ORGNZ-10: Advanced Segment Definitions

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 10 of 10 | **Created:** February 2026 | **Last Updated:** 10/13/2026

Overview

This notebook is a deep-dive into the mechanics of creating effective segment definitions in Dynatrace Grail. While **ORGNZ-08** introduced segments, their structure, and design patterns, this notebook focuses on the practical details: filter condition syntax, data type include rules, metadata enrichment for segments, advanced variable patterns, and troubleshooting techniques.

The content draws from real-world implementation patterns, including host group-based segmentation strategies used in enterprise environments.

Table of Contents

1. Filter Condition Syntax
2. Data Type Include Rules
3. Primary Grail Fields and Enrichment
4. Advanced Variable Patterns
5. Host Group-Based Segments
6. Segment Visibility and Sharing
7. Cross-App Integration
8. Known Limitations and Workarounds
9. Troubleshooting and Performance
10. Segments via the Filter Segments API and Terraform

11. Consuming Segments via the Query API
12. Davis Problem Segment Include Shape

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:filter-segments:read` and `storage:filter-segments:write`
Knowledge	Completed ORGNZ-08 (Grail Segments fundamentals)
Data	At least 1 hour of log, span, and entity data

Learning Objectives

By the end of this notebook, you will:
- Master filter condition syntax including operators, field access, and combining conditions
- Understand data type include rules and the one-include-per-type constraint
- Know which metadata fields propagate across all signals (and which don't)
- Create advanced variable definitions using DQL entity queries
- Build host group-based segments from real-world naming conventions
- Troubleshoot common segment issues and apply performance best practices

1. Filter Condition Syntax

Segment filter conditions define which data a segment includes. Understanding the available operators and field access patterns is critical for building effective segments.

Supported Operators

Operator	Example	Notes
`=` (equals)	`k8s.namespace.name = "production"`	Exact match, best performance
`!=` (not equals)	`k8s.namespace.name != "kube-system"`	Exclude specific values
`starts-with`	`dt.host_group.id = $platform*`	Prefix match with wildcard `*`
`IN`	`k8s.namespace.name IN ("prod", "staging")`	Match any value in list
`NOT IN`	`k8s.namespace.name NOT IN ("kube-system")`	Exclude list of values

> **Important — operator availability depends on data type:**
> - The underlying operator set is `=` **and** `in()` only. `starts-with`, `contains`, and `ends-with` are not separate operators — they are expressed by placing a **wildcard** `*` **inside the value** (`"prefix*"`, `"*text*"`, `"*suffix"`).
> - **Signal data includes** (logs, spans, events, metrics) accept wildcards freely, so all three matching modes are available.
> - **Classic entity includes** (`dt.entity.*`): `entity.name` supports **starts-with only** (`entity.name = "payment*"`); contains and ends-with are not supported on any classic-entity property, and other entity *properties* support exact `equals` only. The Segment limits (DT docs) page: *"starts-with, contains, and ends-with are not supported"*, *"entity.name supports starts-with"*, *"Other properties only support equals"*. Tag conditions test **membership in the tag set** rather than a substring, so verify tag-based entity includes against your own tenant.
> - An asterisk may follow a variable name in a starts-with condition — `foo = $bar*`.
>
> For cross-signal consistency, prefer prefix-based naming conventions (e.g., `prod-web-tier` not `web-tier-prod`) so `starts-with` works everywhere.

Field Access Patterns

Pattern	Example	Use Case
Direct field	`k8s.namespace.name = "production"`	Primary Grail Fields
Tag matching	`matchesValue(tags, "env:production")`	Entity tags
Entity name	`entity.name = "my-service"`	Specific entity
Host group	`dt.host_group.id = "prod-web"`	Host group filtering (signal data)
JSON field	`content$.uri = "/health"`	Nested JSON in log content

Combining Conditions

- **AND** — Multiple conditions within a single include are AND-combined:
  `k8s.namespace.name = "production" AND k8s.cluster.name = "main-cluster"`
- **OR** — Use the `or` keyword within a single include (you cannot have multiple includes for the same data type):
  `k8s.namespace.name = "production" OR k8s.namespace.name = "staging"`

Test Your Filter Conditions

Before creating a segment, always validate your filter conditions with DQL. Run the equivalent filter clause as a DQL query to verify the results match your expectations.

2. Data Type Include Rules

Segments use **includes** to define which data types are filtered. Understanding how includes work is essential for building segments that behave as expected.

One Include Per Data Type

A segment can have **only one include block per data type**. You cannot define two separate include blocks for `logs`. To match multiple conditions, combine them with `OR` within a single include.

Supported Data Types

Data Type	Category	Example Filter
`logs`	Signal data	`k8s.namespace.name = "prod"`
`spans`	Signal data	`service.name starts-with "checkout"`
`events`	Signal data	`event.type = "CUSTOM_INFO"`
`bizevents`	Signal data	`event.provider = "my-app"`
`dt.entity.host`	Classic entity	`tags contains "env:prod"`
`dt.entity.service`	Classic entity	`entity.name starts-with "payment"`
`dt.entity.process_group`	Classic entity	`relationship: runsOn dt.entity.host`
`dt.entity.kubernetes_cluster`	Classic entity	`entity.name = "main-cluster"`

How Conditions Apply Per Data Type

When a segment is active and a user queries a specific data type, **only the matching include rule is injected**:

- Running `fetch logs` → only the `logs` include condition applies
- Running `fetch dt.entity.host` → only the `dt.entity.host` include condition applies
- Entity include rules do **NOT** filter signal data (and vice versa)

> **Critical:** Not all metadata is available on all data types. For example, `java.jar.file` exists only on spans. If you filter a segment by `java.jar.file`, it will only apply to spans — not to logs, metrics, or events. **Always use Primary Grail Fields for cross-signal segment filtering.**

Include Limits

Limit	Value
Maximum includes per segment	20
Include blocks per data type	1
Expressions per filter condition	10

Since each data type gets at most 1 include, a segment can cover up to 20 distinct data types or entity types.

Configuring a segment can itself consume query budget

Easy to miss because it happens before the segment is saved: the **preview** query the segment editor runs
while you build the filter is a real query. *"If the queried data type is billable, the preview query will
contribute to query consumption"*; if the data type is included in your subscription, it costs nothing extra.

The exposure is deliberately bounded — *"all preview queries are subject to strict constraints, including a
limit of 50 records, sampling, and scan limits"* — so this is a note for anyone iterating on segment
definitions against billable data all afternoon, not a reason to avoid the editor. Note the scope: the
documented consumption statement covers **configuration-time previews**, not the cost of a segment once it is
applied to ordinary queries.

> **Sources:** Segments — query consumption (DT docs) — both quoted sentences; read at source 08/27/2026.

3. Primary Grail Fields and Enrichment

The foundation of effective segment definitions is **metadata enrichment**. In Dynatrace's 3rd-generation architecture, each data point is treated independently. Unlike 2nd-gen where signals were tightly coupled to entities via Management Zones, in 3rd-gen every signal (logs, spans, metrics, events) must be enriched with the corresponding metadata.

Primary Grail Fields

Primary Grail Fields are infrastructure-related fields that are **automatically propagated across ALL data types** (spans, logs, metrics, topology). These are the ideal fields for segment filter conditions because they guarantee cross-signal consistency.

Primary Grail Field	DQL Field Name	Source
Host Group	`dt.host_group.id`	OneAgent host group configuration
Kubernetes Cluster	`k8s.cluster.name`	K8s integration
Kubernetes Namespace	`k8s.namespace.name`	K8s integration
AWS Account	`aws.account.id`	AWS integration
Azure Subscription	`azure.subscription`	Azure integration
GCP Project	`gcp.project.id`	GCP integration

> **Best Practice:** Always prefer Primary Grail Fields in segment definitions. They are indexed, automatically enriched, and consistent across all signal types.

What Propagates to Service Metrics

Derived data — service metrics, Davis events and problems — carries primary Grail fields, the special fields and primary Grail tags. Standard tags and signal-local attributes do not:

Field	Propagates to Service Metrics
Primary Grail Fields (`dt.host_group.id` , `k8s.namespace.name` , …)	Yes

Field	Propagates to Service Metrics
`dt.security_context`	Yes
`dt.cost.costcenter`, `dt.cost.product`	Yes
Primary Grail Tags (`primary_tags.<key>`)	Yes
Host tags without the `primary_tags.` prefix, Kubernetes labels, cloud tags	**No**
Signal-local attributes (e.g. OpenTelemetry attributes without the `primary_tags.` convention)	**No**

The Management Zones → Segments upgrade guide still lists only the three special fields as reaching derived data; the primary Grail fields and tags page is broader, and on the validation tenant on 10/06/2026 `dt.service.request.count` split cleanly by two different `primary_tags.*` keys. MZ2POL-05 §5.2 and FAQ-26 §4 cover the conflict and the full reach-by-kind matrix.

> **Sources:** Primary Grail fields and tags (DT docs) — *"Dynatrace enriches all derived signals (service metrics, Davis events, and problems) with the same tags."*, Enrich OpenTelemetry with primary Grail fields and tags (DT docs) — non-convention attributes *"aren't automatically propagated across signal types or to Davis events and problems."*, Best practice examples: from Management Zones to Segments (DT docs) — *"Currently, only keys will be carried over to derived data: dt.security_context, dt.cost.costcenter, and dt.cost.product."*

Primary Grail Tags

When Primary Grail Fields don't align with how your organization wants to segment data (e.g., shared infrastructure with multiple apps on one host group), use **Primary Grail Tags**.

- Identified by the prefix `primary_tags.`
- Example: `primary_tags.stage=prod`, `primary_tags.team=platform`
- Set via host tags, environment variables (`DT_TAGS`), Kubernetes annotations or central enrichment rules, or OpenPipeline — not host properties (see *Enrichment via Host Tags* below)

- Propagated across all data points, similar to Primary Grail Fields

> **Kubernetes coverage:** both central enrichment rules and dedicated annotations (`metadata.dynatrace.com/primary_tags.<key>`) reach Kubernetes platform metrics, Kubernetes events and Smartscape Kubernetes entities, so one `primary_tags.*` include scopes K8s metrics, events, logs and spans alike. On the validation tenant on 10/13/2026, 7,377 of 8,617 Kubernetes events in 24 h carried a `primary_tags.*` field, and `dt.kubernetes.container.restarts` split by it. Two documented caveats apply:
>
> - OneAgent-based enrichment needs the **Cloud Native Full Stack** or **Application Monitoring** deployment mode; Classic Full Stack is not supported.
> - The cost-allocation page says **pod** annotations don't reach Kubernetes metrics, events, Smartscape entities or Prometheus metrics, while the Kubernetes page ticks a combined pod/workload/namespace column. Until the pages agree, set any tag that must scope K8s metrics with a central rule or at namespace level.
>
> **Sources:** Enrich Kubernetes telemetry with primary Grail fields and tags (DT docs) — *"consistently across logs, metrics, spans, events, and Smartscape entities"*; *"OneAgent-based enrichment requires the Cloud Native Full Stack or Application Monitoring deployment mode. Classic Full Stack is not supported."*, Configure cost allocation (DT docs) — *"Pod annotations don't enrich Kubernetes metrics, events, Smartscape entities, or Prometheus metrics."*

Enrichment Approaches

Scenario	Approach	Effort	Granularity
Dedicated Infrastructure	Host group + host tags	Low	Host-level
Shared Infrastructure	`DT_TAGS` environment variable per process	Higher	Process-level
Kubernetes	Primary Grail Fields (automatic)	None	Namespace/cluster
Cloud (AWS/Azure/GCP)	Native cloud tags	None	Account/subscription

Enrichment via Host Tags

For dedicated infrastructure, set these as **host tags** — at OneAgent installation (`--set-host-tag`), with `oneagentctl --set-host-tag` , or via **Deployment Status**:

1. `dt.security_context=<team-or-app>` — For IAM access control
2. `dt.cost.costcenter=<cost-center>` — For cost allocation
3. `dt.cost.product=<product>` — For product tracking
4. `primary_tags.<key>=<value>` — For custom enrichment

The docs name host tags for each of these: *"OneAgent host tags: Add dt.security_context= as a host tag during installation or via Deployment Status."* and *"You can also set dt.cost.costcenter and dt.cost.product directly as host tags during OneAgent installation or via the central primary Grail tag configuration."* For `primary_tags.*` the distinction is now enforced: from OneAgent 1.345, host-level primary tags are derived from host tags only, and values set as host properties stop being promoted (see **ORGNZ-99** row 60a).

> **Sources:** Configure security context (DT docs), Configure cost allocation (DT docs).
>
> **Restart Requirements:**
> - **Logs**: No application restart needed — OneAgent log module auto-enriches after its own restart
> - **Spans**: Application restart **required** to apply enrichment to traces
> - **Metrics**: OneAgent restart applies automatically

Verify Enrichment Coverage

Before building segments, verify that the fields you plan to filter on actually exist on your signal data. Fields with 0% coverage need enrichment via OneAgent configuration or OpenPipeline.

4. Advanced Variable Patterns

Variables make segments dynamic by letting users select values at runtime (e.g., choosing a host group or namespace from a dropdown).

Primary vs Secondary Variables

The variable definition is a DQL query. The columns in the result set determine the variable behavior:

Column Position	Role	Behavior
First column	Primary variable	Shown in the dropdown selector
Additional columns	Secondary variables	Available for use in filter conditions

Variable Rules

Rule	Detail
Maximum values in dropdown	10,000 per variable
Selected values per segment	100 maximum
Wildcard	Allowed **after** variable name: `$cluster*` (starts-with)
No wildcard in names	Variable names and values cannot contain `*`
Permissions	Users must have read access to entities queried by the variable DQL
Empty dropdown	Usually means the user lacks permission for the variable query

> **SaaS 1.347 — staged tenant rollout from 09/16/2026:** segment variable queries move from the consuming app's context to the Segments app's. Once the change reaches your tenant, *"variable queries succeed even when the consuming app lacks the required data scopes"*, provided users hold `ALLOW app-engine:apps:run WHERE shared:app-id = 'dynatrace.segments.management'` — without it, *"variable queries will fall back to the old behavior."* Until then, the two permission rows above describe the working behaviour. Source: What's new in SaaS 1.347 (DT docs).

Variable DQL Examples

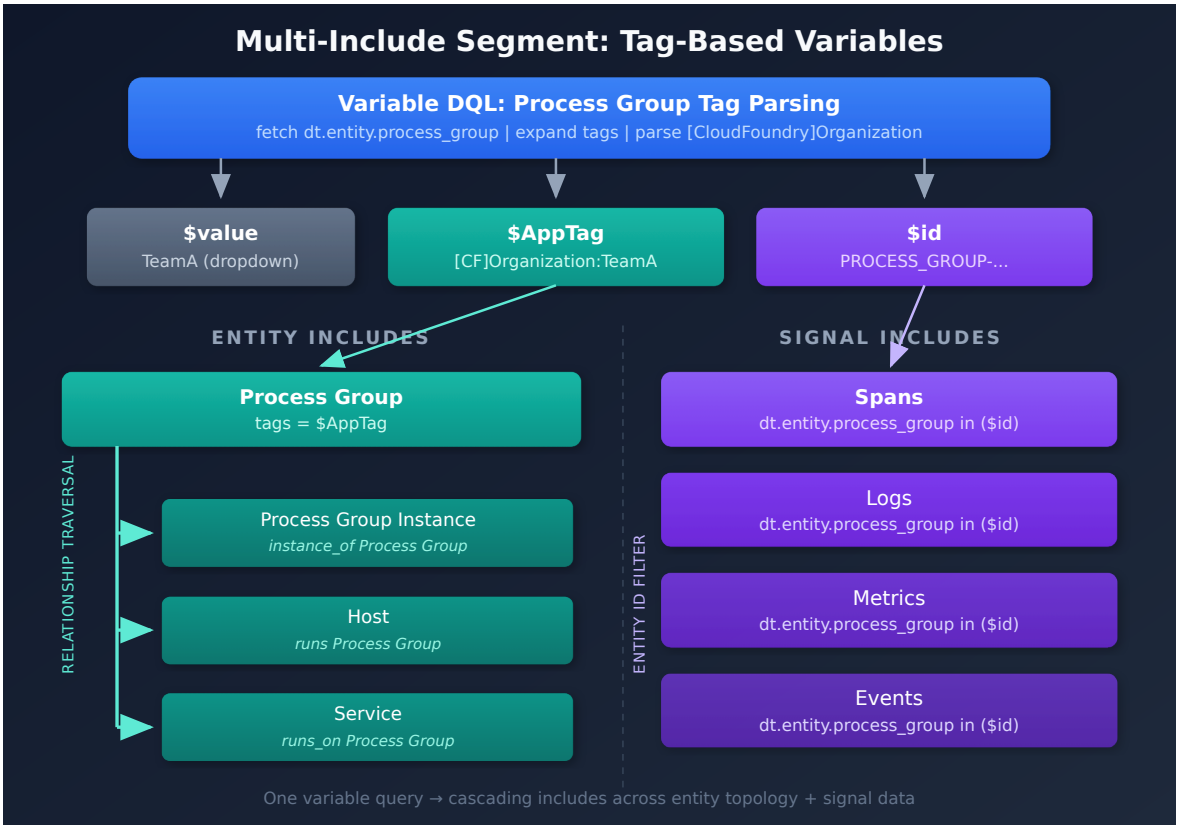
The following queries demonstrate how to create variable definitions for common use cases.

Multi-Include Segment Pattern: Tag-Based Variables

The CloudFoundry query above creates three variables from process group tags:

Variable	Example Value	Use
`$value`	`TeamA`	Display in dropdown
`$id`	`PROCESS_GROUP-1234567`	Filter signal data by entity ID
`$AppTag`	`[CloudFoundry]Organization:TeamA`	Match entity tags

Use these variables across **multiple includes** to build a segment that filters both entity and signal data:



Include Type	Filter	How It Works
Process Group	`tags = $AppTag`	Matches process groups by tag

Include Type	Filter	How It Works
Process Group Instance	`instance_of Process Group`	Relationship traversal — includes instances of matched process groups
Host	`runs Process Group`	Relationship traversal — includes hosts running matched process groups
Service	`runs_on Process Group`	Relationship traversal — includes services on matched process groups
Spans	`dt.entity.process_group in ($id)`	Signal include — filters spans by process group entity ID

> **Key Insight:** This pattern bridges entity and signal data in a single segment. Entity includes use tag matching and relationship traversal to cascade across the topology, while signal includes use the entity ID variable to filter spans, logs, and metrics directly. This is especially powerful when Primary Grail Fields are not available for your use case and you need to segment by application-level metadata like CloudFoundry organizations or custom tags.

Using Variables in Filter Conditions

After defining a variable query, reference the columns in your segment filter:

```
Filter: dt.host_group.id = $host_group OR dt.entity.host_group = $id
```

Where `$host_group` references the primary variable (first column) and `$id` references the secondary variable (second column).

Wildcard with variables — Use `$variable*` to match prefixes:

```
Filter: dt.host_group.id = $platform*
```

This matches all host groups that **start with** the selected platform value.

5. Host Group-Based Segments

A common enterprise pattern is encoding multiple dimensions into host group names using a naming convention. This section shows how to create segments from these encoded dimensions.

Real-World Pattern

Consider a customer using the host group naming convention `<platform>_<app>_<stage>`:

Host Group	Platform	App	Stage
`k8s_multi_prod`	k8s	multi	prod
`k8s_easytravel_staging`	k8s	easytravel	staging
`onPrem_easytravel_staging`	onPrem	easytravel	staging
`onPrem_easytrade_prod`	onPrem	easytrade	prod

Best Practice: One Segment Per Dimension

Create a **separate segment for each dimension** (platform, app, stage). This gives users the flexibility to combine them as needed.

Step 1: Extract Dimensions with DQL

Use DQL to parse the host group name and extract each dimension as a variable:

Step 2: Create Variable DQL for Each Dimension

Platform variable:

```
fetch dt.entity.host_group
| parse entity.name, """LD:platform '_' LD:app '_' LD:stage"""
| dedup platform
| fields platform
```

App variable:

```
fetch dt.entity.host_group
| parse entity.name, """LD:platform '_' LD:app '_' LD:stage"""
| dedup app
| fields app
```

Stage variable:

```
fetch dt.entity.host_group
| parse entity.name, """LD:platform '_' LD:app '_' LD:stage"""
| dedup stage
| fields stage
```

Step 3: Define Filter Conditions

Segment	Filter Condition	Notes
Platform	`dt.host_group.id = $platform*`	Matches host groups starting with selected platform
App	`dt.host_group.id contains $app`	Matches host groups containing selected app (signal includes only)
Stage	`dt.host_group.id ends-with _$stage`	Matches host groups ending with selected stage (signal includes only)

> **Note:** contains and ends-with matching (wildcards anywhere in the value) works on signal data includes. On classic entity includes, `entity.name` supports starts-with only; contains and ends-with are not supported on any classic-entity property, and other entity properties are exact-equals only, so prefix-based naming conventions are the reliable pattern there. Always test with the segment preview before deploying.

Step 4: Validate Across Signal Types

After creating the segments, validate they filter correctly across:
- **Logs** — Check record count reduction when segment is applied
- **Distributed Traces** — Verify spans are filtered by the host group dimension
- **Hosts / Services** — Confirm entity filtering matches expectations
- **Infrastructure** — Verify metrics are scoped correctly

> **Tip:** The host group is a Primary Grail Field, so it propagates across all signal types. This makes it an excellent candidate for segment filtering.
>
> **K8s analog.** For Kubernetes-heavy tenants, the equivalent Primary Grail Fields are `k8s.cluster.name` (cluster-scoped segment) and `k8s.namespace.name` (namespace-scoped segment). The canonical Segment Kubernetes clusters (DT docs) walkthrough uses exactly this pattern — one filter condition (e.g. `k8s.cluster.name = "gke-klu"`) applied across signals and entities. The Kubernetes-namespace entity type is `dt.entity.cloud_application_namespace` (not the more intuitive-sounding `dt.entity.kubernetes_namespace` — which does not exist).

6. Segment Visibility and Sharing

Visibility Settings

Setting	Behavior
Unlisted (default)	Visible only in the owner's list of segments
Anyone in the environment	Listed in everyone's segment list in the environment

Permissions Model

Permission	Action	Included In
`storage:filter-segments:read`	View and use segments	Standard User, Pro User, Admin User
`storage:filter-segments:write`	Create and edit segments	Standard User, Pro User, Admin User
`storage:filter-segments:share`	Share with others	Pro User, Admin User
`storage:filter-segments:delete`	Delete segments	Standard User, Pro User, Admin User

Permission	Action	Included In
`storage:filter-segments:admin`	Maintain all segments in the environment, including ones you do not own	Admin User

> **Sources:** Dynatrace default policies reference (DT docs) — Standard User: *"ALLOW storage:filter-segments:read, storage:filter-segments:write, storage:filter-segments:delete;"*; Pro User adds `storage:filter-segments:share`; Admin User adds `storage:filter-segments:admin`. dynatrace_segment resource (Dynatrace GitHub) — *"Maintain all Filter-Segments on the environment"*.

Governance Recommendations

- **Platform team** owns environment and infrastructure segments and sets them to *Anyone in the environment*
- **Application teams** own app-specific segments, kept *Unlisted*, and surface them by reference from team-owned notebooks or dashboards (there is no direct "share with group" mechanism on the segment itself — sharing flows through the containing document)
- Maintain a small set of "official" environment-wide segments
- Use consistent naming: `team-`, `env-`, `app-`, `region-` prefixes

> **Visibility ≠ access control.** The visibility setting only affects whether a segment is listed in other users' segment pickers. **Anyone with `storage:filter-segments:read` can apply any segment they know the ID of** — IAM policies on the underlying data (logs/spans/buckets) are what actually gate what they see when the segment is applied.
>
> **Sources:** Visibility of segments (DT docs) — verbatim: *"the Visibility setting doesn't affect general access to segments"* and *"Unlisted segments can still be made available to others by being referenced in apps, such as in shared notebooks and dashboards."*

7. Cross-App Integration

Cross-App Persistence

When you select a segment in one Dynatrace app (e.g., Logs), the selection **persists** as you navigate to other apps (e.g., Distributed Traces, Problems). This means you don't need to reapply filters when drilling into different data types during an investigation.

App Support Matrix

App	Segment Support	Notes
Dashboards	Full	Dashboard-level and tile-level segments
Notebooks	Full	Segment selector at top of notebook
Logs	Full	Filters all log queries
Distributed Traces	Full	Filters span queries
Problems	Full	Requires events include with `event.kind = "DAVIS_PROBLEM"`
SLOs	Full	Filters SLO evaluation scope
Site Reliability Guardian	Full	Filters validation scope
Workflows	Full	Available in workflow DQL steps
Smartscape	Full	Filters topology view

Dashboard Segment Behavior

- **Dashboard-level segment** applies to all tiles
- **Tile-level segment** overrides the dashboard segment for that specific tile
- Use this pattern for executive dashboards that show cross-team metrics alongside team-specific views

8. Known Limitations and Workarounds

Limitation	Detail	Workaround
detected problems require event includes	To filter problems with segments, define an events include with `event.kind = "DAVIS_PROBLEM"`; entity includes alone do not filter problem records	Add event-type includes alongside entity includes
Single relationship traversal	Entity relationships can only traverse one hop	Target specific entity types directly in includes
Inclusions only, no exclusions	Cannot say "everything EXCEPT team-X"	Explicitly include what you want (MZ supported exclusions; segments do not)
Entity properties are equals-only	On classic entity includes, `entity.name` supports starts-with only; contains and ends-with are not supported on any classic-entity property, and other properties support exact `equals` only	Use prefix-based naming conventions; put the broader condition on a signal include
1 include per data type	Cannot have two `logs` include blocks	Combine conditions with `OR` within a single include
Max 20 includes per segment	Hard limit on rule count	Consolidate rules; use variables for flexibility
Max 10 expressions per filter	Limits filter complexity	Use OpenPipeline to pre-enrich data with simpler filter fields
Max 10 segments per query	Cannot stack unlimited segments	Design broader segments instead of many narrow ones
Variable dropdown empty	Users without entity read permissions see empty dropdown	Ensure variable DQL targets entities the user can read

9. Troubleshooting and Performance

Common Issues

Symptom	Likely Cause	Fix
Empty variable dropdown	Missing read permissions for the data the variable DQL queries	Grant `storage:entities:read` (and `storage:<table>:read` for any other data object the variable DQL reads). Once SaaS 1.347 reaches your tenant, also `app-engine:apps:run` on `dynatrace.segments.management` (§4)
Segment shows no data	Filter field not enriched on signal data	Verify enrichment with audit queries from Section 3
Segment filters entities but not logs	Entity includes don't apply to signals	Add separate signal-type includes (logs, spans)
Unexpected data included	OR logic combining values	Review filter condition logic within the include
Segment missing in app	Segment set to unlisted and not shared	Share segment or set visibility to public
Segment works for logs but not spans	Field exists on logs but not spans	Use Primary Grail Fields for cross-signal consistency

Performance Best Practices

Every segment filter condition is injected into every query the user runs. Keep conditions simple:

1. **Prefer Primary Grail Fields** — They are indexed and optimized for filtering
2. **Use exact matches** — `=` is faster than `starts-with`
3. **Minimize expressions** — Fewer conditions = better query performance
4. **Use OpenPipeline pre-processing** — If conditions would be complex, pre-enrich a simple field (like `dt.security_context`) and filter on that instead

5. **Narrow scope first** — Start with the most restrictive condition in AND combinations

Naming Conventions

Prefix	Use Case	Examples
`team-`	Team-based segments	`team-platform-infra`, `team-checkout`
`env-`	Environment segments	`env-production`, `env-staging`
`app-`	Application segments	`app-checkout-full`, `app-payments`
`region-`	Regional segments	`region-us-east`, `region-eu-west`

Segment Lifecycle

1. **Design** — Identify data types, filter conditions, variables
2. **Test** — Validate filter logic with DQL queries
3. **Create** — Build in the Segments app
4. **Share** — Distribute to appropriate users/groups
5. **Monitor** — Check for empty results, permission issues
6. **Retire** — Remove when no longer needed; review quarterly

Discover Available Filter Candidates

DQL: Enrichment Coverage Audit

Audit primary Grail field population before building segment definitions:

```
// Audit primary Grail field coverage on logs — low-coverage fields need
enrichment before segment use
fetch logs, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_security_ctx = countIf(isNotNull(dt.security_context))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100, decimals:
1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100,
decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100,
decimals: 1),
    sec_pct = round(toDouble(with_security_ctx) / toDouble(total) * 100,
decimals: 1)
```

10. Segments via the Filter Segments API and Terraform

Segments authored in the Segments app are convenient for exploration, but **production-grade segment libraries should be managed as code**. Manual segments drift, get orphaned when their creator leaves, and lack the review/promotion discipline that buckets and IAM policies already have in mature tenants.

> **Corrected 09/24/2026.** Earlier versions of this section described segments as Settings 2.0 objects under a `builtin:filter-segments` schema, created with `POST /api/v2/settings/objects`. That schema does not exist — the Settings API answers 404 for it — so that call cannot create a segment. Segments have their own platform API.

The Filter Segments API

Segments are managed through the **Filter Segments API** at `/platform/storage/filter-segments/v1/filter-segments`, authenticated with a platform token. The body carries the same fields you fill in the UI:

Field	Purpose
`name`	Segment name shown in the UI (the API addresses a segment by its generated `uid`)
`description`	Free-text purpose statement
`isPublic`	`false` — *"The filter-segment is private and only visible to the owner"*; `true` — *"The filter-segment is visible to everyone in the environment"* (needs `storage:filter-segments:share`)
`includes`	Array of `{ "dataObject": …, "filter": … }` — one per data object (see §2)
`variables`	Optional DQL-backed variable definition (see §4): `{ "type": "query", "value": "<DQL>" }`

For `dataObject`, *"Use '_all_data_object' to apply it to all dataObjects."* The `filter` is the segment's filter expression as the Segments app stores it — on the validation tenant (read 09/24/2026) the Dynatrace ready-made segments store plain text such as `dt.host_group.id = $dt.host_group.id`, while the SDK's own create example carries a serialized filter tree. Rather than hand-writing that value, build the segment once in the UI, read it back, and version the JSON you get. With `$DT_ENV` set to `https://<environment-id>.apps.dynatrace.com`, the first call lists segments (`uid`, `name`, `isPublic`, `owner`); the second returns one segment with its includes and variables — the body to keep in version control:

```
curl -s "$DT_ENV/platform/storage/filter-segments/v1/filter-segments" \
  -H "Authorization: Bearer $DT_PLATFORM_TOKEN"

curl -s "$DT_ENV/platform/storage/filter-segments/v1/filter-
segments/&lt;uid&gt;?add-fields=INCLUDES&amp;add-fields=VARIABLES" \
  -H "Authorization: Bearer $DT_PLATFORM_TOKEN"
```

Create a segment by sending a body of that shape with `POST` to the collection path — the SDK's `createFilterSegment`, which needs `storage:filter-segments:write`, plus `storage:filter-segments:share` when `isPublic` is `true`.

> **Auth scheme:** Platform Tokens (`dt0s16.*` / `dt0s01.*`) require the `Bearer` header; classic API Tokens (`dt0c01.*`) require `Authorization: Api-Token`. Mixing schemes returns 401 even with all the right scopes — see AUTOM-04 §3 for the routing rules.

Manage via Terraform

The `dynatrace-oss/dynatrace` Terraform provider exposes segments through the `dynatrace_segment` resource, whose top-level attributes (`name`, `description`, `is_public`, `includes`, `variables`) mirror the API body above. The resource documentation asks for an **OAuth client** — *"please define the environment variables `DT_CLIENT_ID`, `DT_CLIENT_SECRET`, `DT_ACCOUNT_ID` with an OAuth client"* — carrying these permissions:

- `storage:filter-segments:read` and `storage:filter-segments:write` for create/update
- `storage:filter-segments:share` if the segment's visibility is set beyond the owner (there is no share-with-specific-group mechanism — see §6)
- `storage:filter-segments:delete` if `terraform destroy` should remove segments
- `storage:filter-segments:admin` (*"Maintain all Filter-Segments on the environment"*) when the provider must manage segments it does not own

It adds that *"Depending on the segment configuration, additional storage permissions may be required for DQL-related access"* — for example `storage:logs:read` or `storage:entities:read` for a segment whose variable DQL reads those data objects.

See AUTOM-04 §6 for the resource-table-driven catalog (including which auth scheme each Terraform resource requires) and AUTOM-09 for the broader GitOps repo layout and state-backend recommendations.

Platform tokens. The provider accepts `DYNATRACE_PLATFORM_TOKEN` and documents that *"Platform tokens can't be used for IAM (Account Management) or classic resources"* — `dynatrace_segment` is neither. But the segment resource page documents only the OAuth-client path, so treat a platform token as untested for this resource and confirm it in a non-production plan/apply before relying on it. If the same configuration also holds classic API-token resources, setting `DYNATRACE_HTTP_OAUTH_PREFERENCE` to `true` means *"the provider will favor platform or OAuth tokens over API tokens."*

When IaC is worth the overhead

No Dynatrace documentation prescribes when a segment should move from the UI to code. In community practice, teams draw the line roughly as follows — adapt it to your own review process.

Scenario	Author in UI	Manage as code
One-off investigation segment used by a single engineer	Yes	No
Standing "official" segments shared across teams (env, team, app)	No	Yes
Segments referenced from dashboards or workflows that are themselves in code	No	Yes
Segments enforcing a compliance scope (segment + bucket + policy triplet)	No	Yes
Segments whose `variables` block contains DQL the team is iterating on	Yes (then promote)	Yes

The split this adds up to: **exploration in the UI, promotion to code** the moment a segment becomes load-bearing for someone else's workflow.

Drift between UI and code

When a segment is managed in code but someone edits it through the UI, the next `terraform apply` (or API update) reverts the change. Two defenses:

1. **Document ownership** — add a `description` like `Managed in Terraform repo X — open a PR, do not edit here`.
2. **Restrict write scope** in production: only the IaC service user holds `storage:filter-segments:write` for public segments; everyone else gets `read` + private-segment authoring.

> **Sources:** Get started with segments — analyze monitoring data (DT docs), Grail storage filter-segments SDK (Dynatrace Developer). dynatrace_segment resource (Dynatrace GitHub), Terraform provider authentication (Dynatrace GitHub). Endpoint paths read from the validation tenant 09/24/2026.

11. Consuming Segments via the Query API

A common point of confusion: **there is no inline DQL syntax that references a saved segment by name.** You will not find a `loadSegments:` argument on `fetch`, nor a `| filter <segment-name>` pipe. Segments are applied as **query context**, passed alongside the DQL string in the Grail Query API request body.

Request shape

Segments attach to the `/platform/storage/query/v1/query:execute` request body as a `filterSegments` array. Each element references a segment by its `id` (the segment's `uid`) and can supply values for its variables:

```
POST /platform/storage/query/v1/query:execute
Authorization: Bearer &lt;platform-token&gt;
Content-Type: application/json

{
  "query": "fetch logs, from:-1h | summarize count = count(), by:{loglevel}",
  "filterSegments": [
    { "id": "&lt;segment-id&gt;", "variables": [ { "name": "stage", "values":
["prod"] } ] }
  ]
}
```

> **Corrected 09/24/2026.** Earlier versions showed a `segments: [{ "segmentId": … }]` field. The query service does not define that field, so a request built from it runs **unfiltered** — IAM still applies, but the segment does not.

The Grail query engine merges the segment's include filters into the executing query before scanning — same effect as if the filters had been hand-written into the DQL.

Where the API call happens

You almost never invoke the Query API directly. The Dynatrace surfaces that read and write to Grail pass `filterSegments` for you, transparently:

Surface	How segments are passed
Notebooks	Segment selector at top of notebook is injected into every cell's API call
Dashboards	Dashboard-level selector + per-tile override; injected into each tile's API call
Logs, Distributed Traces, Problems, SLOs apps	App's segment selector is included in every Grail call the app makes
Workflows DQL task	Task config carries a segments array; injected at execution time
Site Reliability Guardian validators	Validator config attaches segments to its evaluation queries
Direct API consumers (custom apps, scripts, terraform-provider-dynatrace queries)	**You** must add the `filterSegments` array to the request body

Implication for custom integrations

If you are writing a script that queries Grail through the API — for example, a Workflow JavaScript task using `@dynatrace-sdk/client-query`, a CI/CD job emitting a Site Reliability Guardian-style check, or a custom Dynatrace app — and you want it to honor a saved segment, **you must pass the `filterSegments` array yourself**. The DQL string alone will not pick up "the currently active segment in the app" — there is no such thing outside a UI session. In community practice, the most common cause of "my segment works in the app but not in my script" is the script's API call missing the `filterSegments` array — verify against your own integrations.

Variables in segments — how the value is supplied

If a segment defines variables (§4), the consumer must also supply the **selected values** for those variables. In the UI this is the dropdown selection; in the API each `filterSegments` element carries `variables: [{ "name": …, "values": [ … ] }]`, and *"The values defined in the request will replace the variables used in the filter segments."* Several segments in one request narrow together: *"If the request defines multiple segments, they will be combined using the AND boolean operator, and the Grail data will be filtered accordingly."*

Verified on the validation tenant 09/24/2026: `fetch logs, from:-1h | summarize n = count(), by:{k8s.namespace.name}` returned `easytrade`, `hipstershop`, `online-boutique` and more without a segment, and only `dynatrace` with the ready-made `k8s.namespace.name` segment passed as `filterSegments: [{ "id": "<uid>", "variables": [{ "name": "k8s.namespace.name", "values": ["dynatrace"] }] }]`.

Why this matters for design

Because segments are query context, **a segment's effective behavior depends on who is calling**. The same segment applied by a user with `storage:logs:read` on bucket A returns rows from bucket A; applied by a user without that scope, the same segment returns nothing from bucket A. Segments do not bypass IAM — they layer on top of it. This is the §9 "Segments filters entities but not logs" troubleshooting story restated as a design principle.

> **Sources:** Segments in DQL queries (DT docs), Grail service overview (Dynatrace Developer), Grail storage query SDK (Dynatrace Developer).

12. Davis Problem Segment Include Shape

The Problems app shows up as "Full" support in the cross-app integration table in §7, but with a critical asterisk noted in §8: **entity includes alone do not filter problem records**. The Problems app documentation is explicit: *"Since problems are stored as events in Grail, segments created for filtering problems must define an event filter."* Its own example is `cloud.region = "us-east-1c" AND event.kind = "DAVIS_PROBLEM"`.

Include shape

Data type	Filter	Effect
`events`	`event.kind = "DAVIS_PROBLEM"` AND your scoping condition	Filters problem records to the team / env / app scope

Data type	Filter	Effect
`dt.entity.host` (optional)	scoping condition on the same dimension	Not applied to problems — scopes host data when the same segment is used in other apps
`dt.entity.service` (optional)	scoping condition on the same dimension	Not applied to problems — scopes service data when the same segment is used in other apps

Worked example — production problems only

```
name: env-production-problems
description: "Production-scoped Davis problems and the affected entities"
visibility: PUBLIC

includes:
  - dataType: events
    filter: 'event.kind = "DAVIS_PROBLEM" AND dt.host_group.id starts-with
"prod-"'

  - dataType: dt.entity.host
    filter: 'tags = "environment:production"'

  - dataType: dt.entity.service
    filter: 'tags = "environment:production"'
```

In the Problems app, only the `events` include does any work: *"Segment filters are directly applied to the problem Grail records. Consequently, no entity filters are applied to the problem unless the entity ID is chosen as a primary field of the filtered problem."* Drop the `events` include and the problem feed stops filtering — entity includes alone are not enough.

The entity includes are optional and earn their place outside the Problems app. Because a segment selection carries across apps (§7), in community practice teams add them so that one "production" segment also scopes the host and service views an investigation moves into next — verify the behavior in the apps your teams actually use.

Why this works

Davis problems live in the `events` data object with `event.kind = "DAVIS_PROBLEM"`. The Problems app issues DQL against that data, and *"During query execution, Grail evaluates only relevant conditions of segments passed based on the query's targeted data object"* — so a query over `events` reads the `events` include and ignores the entity includes.

Validating the include

Before saving the segment, run the equivalent DQL to make sure the filter produces the problem set you expect. Count problems on `dt.davis.problems`, which holds one record per problem. The `events` data object holds one record per problem *update*, so `count()` there over-reports — 15,735 records for 465 problems over 24 hours on a live tenant (10/05/2026).

```
// One record per problem (dt.davis.problems), so count() counts problems
fetch dt.davis.problems, from:-24h
| filter event.kind == "DAVIS_PROBLEM"
| filter startsWith(dt.host_group.id, "prod-")
| summarize problems = count(), by:{event.status}
```

If the count is zero, the underlying scoping field probably isn't enriched on problem events in your tenant — fall back to a Primary Grail Field that is (§3 audit query).

> **Sources:** Problems app (DT docs), Segments in DQL queries (DT docs).

Summary

In this notebook you learned:

1. **Filter condition syntax** — Operators (`=` , `!=` , `starts-with` , `contains`), field access, AND/OR logic
2. **Data type include rules** — One include per type, conditions scoped to queried data type
3. **Primary Grail Fields** — Which metadata propagates across all signals (and which doesn't)

4. **Enrichment approaches** — Dedicated vs shared infrastructure, host tags vs DT_TAGS
5. **Advanced variables** — Primary/secondary, DQL queries, permission requirements
6. **Host group-based segments** — Parsing naming conventions, one segment per dimension
7. **Visibility and sharing** — Public vs unlisted, governance model
8. **Cross-app integration** — Persistence, dashboard vs tile-level segments
9. **Limitations and troubleshooting** — detected problem event includes, entity operator restrictions, performance tips
10. **Segments via the Filter Segments API and Terraform** — `/platform/storage/filter-segments/v1/filter-segments` (`name` , `isPublic` , `includes[{dataObject, filter}]`), the `dynatrace_segment` Terraform resource, UI-vs-code decision criteria, drift management
11. **Consuming segments via the Query API** — `filterSegments` array (`id` + `variables`) on `/platform/storage/query/v1/query:execute` ; no inline DQL syntax; custom integrations must pass segments themselves
12. **Davis problem segment include shape** — Why entity includes alone are not enough; the load-bearing `events` include with `event.kind = "DAVIS_PROBLEM"`

Series Summary

Notebook	Topic	Key Takeaway
ORGNZ-01	Introduction	Three pillars of data organization
ORGNZ-02	Grail Buckets	Bucket fundamentals and limits
ORGNZ-03	Bucket Strategy	Naming, retention, design patterns
ORGNZ-04	Permissions Overview	Permission levels and policy structure
ORGNZ-05	Bucket-Level Access	IAM policies for bucket isolation

Notebook	Topic	Key Takeaway
ORGNZ-06	Security Context	Fine-grained access with dt.security_context
ORGNZ-07	Advanced Permissions	Record and field-level patterns
ORGNZ-08	Grail Segments	Segment fundamentals and design patterns
ORGNZ-09	Enterprise Patterns	Combined approaches at scale
ORGNZ-10	**Advanced Segment Definitions**	**Filter syntax, enrichment, variables, IaC, Query API, Davis problems, troubleshooting**

References

- Grail Segments overview (DT docs)
- Include data in segments (DT docs)
- Variables in segments (DT docs)
- Visibility of segments (DT docs)
- Segment Kubernetes clusters — use case (DT docs)
- Segment limits (DT docs)
- Segments in DQL queries (DT docs)
- Supported data types in segments (DT docs)
- Get started with segments — analyze monitoring data (DT docs)
- Grail storage filter-segments SDK (Dynatrace Developer)
- Grail storage query SDK (Dynatrace Developer)
- Grail service overview (Dynatrace Developer)
- Problems app (DT docs)

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.